

Contents

guide Training language models to follow instructions with human feedback	1
0. 这篇论文到底改变了什么	1
1. 回到 2022 年的语境	2
2. 原论文阅读地图	2
3. 三阶段总图	3
4. 数据从哪里来	3
5. Stage 1: SFT 到底做什么	4
6. Stage 2: Reward Model	4
7. Bradley-Terry Loss	5
8. Stage 3: PPO 为什么要加 KL	5
9. PPO-ptx 是什么	6
10. 四模型视角	7
11. 张量级别走一遍	7
12. 实验证据链	8
13. 为什么这篇论文能成为范式	8
14. 局限和后续问题	9
15. 与 DPO 的关系	9
16. 与本仓库代码怎么对上	9
17. 极简代码: 从偏好对到 PPO reward	10
18. 现在为什么还要读	10
19. 常见误区	11
20. 闭卷掌握检查	11
21. 用 AI agent 学这篇的正确方式	11

guide Training language models to follow instructions with human feedback

原论文: Training language models to follow instructions with human feedback
本地原文 PDF: [learning/rlhf-classic/paper/01_instructgpt.pdf](#)
作者: Ouyang et al.
年份: 2022
类型: paper

0. 这篇论文到底改变了什么

InstructGPT 把大语言模型从 “会续写互联网文本” 推向 “会按用户意图完成任务”。它的核心不是发明一个新的 Transformer layer, 而是把人类反馈变成一条可训练流水线:

```
prompt distribution
-> human demonstrations
-> SFT policy
-> human rankings of model outputs
-> reward model
-> PPO with KL control
-> InstructGPT
```

这篇论文最重要的结论非常有历史分量: 在 OpenAI API prompt distribution 上, 1.3B InstructGPT 的输出可以被标注者偏好于 175B GPT-3 的输出。也就是说, 模型尺寸不是 instruction following 的唯一答案; 数据、偏好和优化目标可以改变用户体验。

它也是现代 RLHF 的经典三段式范式:

1. SFT: 用人工示范教模型 “回答指令”。
2. RM: 用人工排序训练一个 reward model。
3. PPO: 用 reward model 优化 policy, 同时用 KL penalty 防止 policy 跑离 SFT 太远。

后来的 ChatGPT、DPO、RLAIF、constitutional AI、preference optimization、process reward 等路线都可以看成是在改这条流水线的某个环节: 数据怎么来、偏好怎么建模、RL 是否必要、KL 如何控制、reward hacking 怎么防。

1. 回到 2022 年的语境

GPT-3 证明了 scaling 的力量, 但它默认训练目标是 next-token prediction。这个目标会让模型学到 “互联网上接下来可能出现什么文字”, 但用户真正想要的是:

- 遵守 instruction。
- 不胡编乱造事实。
- 避免有害、冒犯或危险输出。
- 在多任务场景下保持有用。
- 能理解隐含约束, 例如 “简短” “不要泄露个人信息” “用表格回答”。

当时已有的路线包括 prompt engineering、few-shot prompting、instruction tuning、summarization preference learning。InstructGPT 的关键推进是把这些经验统一成一条可扩展的产品级 pipeline: 真实 API prompt 分布 + 标注者示范 + 标注者排序 + reward model + PPO。

论文也非常清醒地承认: 这不是把模型对齐到抽象的 “人类价值”。它对齐的是一组标注者和研究者在特定任务说明下表达出来的偏好。这个限制很重要, 因为它直接引出了后续安全研究里的 representativeness、bias、refusal policy、评价协议和多群体偏好问题。

2. 原论文阅读地图

建议按下面顺序读:

1. Abstract: 先抓住 1.3B InstructGPT 胜过 175B GPT-3 的主结论。
2. Figure 1: 看 model size、SFT、PPO、PPO-ptx 的人类偏好曲线。
3. Figure 2: 看三阶段方法图, 这是整篇论文的骨架。
4. Section 3.2: 看 prompt 和数据来源。
5. Section 3.5: 看 SFT、RM、PPO 的训练细节和两个关键公式。
6. Section 4.1: 看 API prompt distribution 上的主结果。
7. Section 4.2: 看 TruthfulQA、toxicity、bias 和 public NLP regressions。
8. Section 5: 看局限, 尤其是 “对齐到谁的偏好”。
9. Appendix C/E: 看超参数、RM 过拟合、PPO-ptx、KL coefficient 的工程细节。

如果时间只够 40 分钟, 至少读 Figure 2、RM loss、PPO objective、Section 4.1 的人评结果。掌握这四块, 就能理解现代 RLHF 论文大多在接哪根线。

3. 三阶段总图

```
base GPT-3
|
| Stage 1: SFT
|   data: prompt x, human-written response y_demo
|   loss: next-token NLL on response tokens
v
supervised policy pi_SFT
|
| Stage 2: reward modeling
|   data: prompt x, K model completions ranked by labeler
|   train: r_theta(x, y) as scalar reward
v
reward model RM
|
| Stage 3: PPO
|   init policy from pi_SFT
|   reward = RM score - beta * KL(pi_RL || pi_SFT)
|   optional: add pretraining gradients
v
InstructGPT, usually PPO-ptx in this paper
```

要注意三段的学习信号不同:

- SFT: 学习信号是人写的正确回答, 数据单位是 (prompt, response), 学到的是指令形式、回答风格和基本任务能力。
- RM: 学习信号是人对多个回答的排序, 数据单位是 (prompt, chosen, rejected), 学到的是标注者偏好的隐式评分函数。
- PPO: 学习信号是 reward model 给分加 KL 约束, 数据单位是 rollout response, 学到的是更偏向高 reward 行为的 policy。

SFT 是 imitation。RM 是 preference modeling。PPO 是 policy optimization。把这三件事混成 “RLHF” 容易丢掉关键细节。

4. 数据从哪里来

论文用了三类 prompt 数据:

- SFT dataset: 约 13k training prompts, 包含 API prompt 和 labeler-written prompt。
- RM dataset: 约 33k training prompts, 标注者对多个模型输出做排序。
- PPO dataset: 约 31k training prompts, 只用 prompt, 不需要人工标签, PPO rollout 时由 RM 打分。

标注者规模约 40 人, 来源包括 Upwork 和 Scale AI。标注者不是随便招来的, 他们经过筛选, 包括敏感内容识别、排序一致性、敏感 prompt 示范写作等维度。这个细节说明 RLHF 不是 “把数据丢给众包” 这么简单, 它把价值判断、任务说明、标注培训和评价协议都放进了训练闭环。

一个容易忽略的点: 训练时标注者主要优先 helpfulness, 而最终评价时更强调 truthfulness 和 harmlessness。这制造了一个早期 RLHF 的核心张力: 用户帮助性、安全性和真实性并不总是一致。

5. Stage 1: SFT 到底做什么

SFT 的输入是:

```
prompt: [p1, p2, ..., pm]
response: [y1, y2, ..., yn]
```

训练时通常把 prompt 和 response 拼起来, 但 loss 只算 response token:

```
input_ids = prompt_ids + response_ids
labels     = [-100 ... -100] + response_ids
```

```
L_SFT = - sum_t log pi(y_t | prompt, y_{<t})
```

-100 只是 PyTorch cross entropy 的 ignore index。它表达一个训练选择: prompt 是条件, 不是模型需要复述的答案。

本仓库对应代码:

```
def sft_loss(logits, labels):
    shift_logits = logits[..., :-1, :]
    shift_labels = labels[..., 1:]
    return F.cross_entropy(
        shift_logits.reshape(-1, shift_logits.size(-1)),
        shift_labels.reshape(-1),
        ignore_index=-100,
    )
```

论文里 SFT 不只是训练 1 个 epoch。作者发现 validation loss 可能 1 epoch 后过拟合, 但更多 epoch 反而提升 RM score 和人类偏好评分。这个结论很有启发: 对齐任务里, 传统语言建模 validation loss 不一定是最终用户偏好的最佳代理指标。

6. Stage 2: Reward Model

Reward model 接收一个 prompt 和一个 completion, 输出一个标量:

```
r_theta(x, y) -> scalar
```

结构上可以理解为:

```
tokens = concat(prompt, completion)
hidden_states = Transformer(tokens)
last_hidden = hidden state at final non-pad token
reward = linear(last_hidden) -> scalar
```

本仓库的 RewardModel 就是这个骨架:

```
class RewardModel(nn.Module):
    def __init__(self, base_lm):
        super().__init__()
        self.lm = base_lm
        self.v_head = nn.Linear(base_lm.config.hidden_size, 1)

    def forward(self, input_ids, attention_mask):
        out = self.lm(input_ids, attention_mask=attention_mask,
                       output_hidden_states=True)
```

```

h = out.hidden_states[-1]
last_idx = attention_mask.sum(-1) - 1
last_h = h[torch.arange(len(h)), last_idx]
return self.v_head(last_h).squeeze(-1)

```

为什么是标量？因为 PPO 需要一个 reward。人类不直接给每个 token 的 reward，人类更容易比较“这个回答比那个回答好”。RM 把这种比较压缩成可优化的分数。

7. Bradley-Terry Loss

对同一个 prompt x ，标注者看到 K 个候选回答并排序。任意一对回答可以变成：

```

y_w = preferred completion
y_l = less preferred completion

```

Reward model 希望：

```

r_theta(x, y_w) > r_theta(x, y_l)

```

Bradley-Terry 形式把 reward 差值变成偏好概率：

```

P(y_w preferred over y_l | x)
= sigmoid(r_theta(x, y_w) - r_theta(x, y_l))

```

loss 是：

```

L_RM = - log sigmoid(r_w - r_l)

```

批量形式：

```

def bt_loss(r_chosen, r_rejected):
    return -F.logsigmoid(r_chosen - r_rejected).mean()

```

张量级别：

```

chosen_ids:    [B, T]
rejected_ids:  [B, T]
r_chosen:      [B]
r_rejected:    [B]
diff:          [B]
loss:          scalar

```

论文的一个工程细节很关键：如果标注者对 $K=4..9$ 个回答排序，直接把所有 pair 打散会导致同一 prompt 下的 pair 高度相关，RM 容易过拟合。作者把同一 prompt 的所有 pair 作为一个 batch element 来处理，这既减少重复 forward，又缓解过拟合。

论文最终主要用 6B reward model，而不是 175B RM。原因不是 175B 一定不准，而是 175B RM 训练更不稳定，且作为 PPO value function 初始化会显著增加计算负担。这里体现了 RLHF 的工程本质：最优组件不是单看分数，还要看稳定性和系统成本。

8. Stage 3: PPO 为什么要加 KL

PPO 阶段把 SFT policy 继续优化。环境很简单，是一个 contextual bandit：

```

sample prompt x
policy generates response y

```

```
RM scores r_theta(x, y)
episode ends
```

如果只最大化 RM 分数, policy 会学习 reward model 的漏洞, 也就是 reward hacking。比如生成 RM 偏好的表面模式, 而不是真的对用户更有帮助。

所以论文给每个 token 加 KL penalty, 使 RL policy 不要偏离 SFT policy 太远:

```
reward_total
    = r_theta(x, y)
    - beta * log( pi_RL(y | x) / pi_SFT(y | x) )
```

更细到 token:

```
KL_t approx = log pi_RL(y_t | prefix) - log pi_SFT(y_t | prefix)
token_reward_t = - beta * KL_t
final token also gets + RM_score
```

本仓库 build_token_rewards 就是这个结构:

```
def build_token_rewards(raw_rewards, response_mask, log_p_act, log_p_ref, beta=0.02):
    kl = log_p_act - log_p_ref
    rewards = -beta * kl
    for b in range(rewards.size(0)):
        idx = response_mask[b].nonzero(as_tuple=False).flatten()
        if idx.numel() > 0:
            rewards[b, idx[-1]] += raw_rewards[b]
    return rewards * response_mask
```

这里 log_p_ref 来自 frozen SFT model。这个 reference model 是 RLHF 训练的锚点。

9. PPO-ptx 是什么

论文发现 PPO 会让 public NLP datasets 出现 performance regressions。直觉上, RLHF 把模型推向“标注者喜欢的 API assistant 行为”, 可能损伤一些原始预训练能力, 尤其是传统 QA、reading comprehension、translation 等 benchmark。

PPO-ptx 的做法是在 PPO 梯度里混入预训练分布的 log-likelihood 梯度:

```
objective =
    E_{x,y ~ D_pi_RL} [
        r_theta(x, y)
        - beta * log(pi_RL(y|x) / pi_SFT(y|x))
    ]
    + gamma * E_{x ~ D_pretrain} [ log pi_RL(x) ]
```

其中:

- beta: 控制 KL penalty 强度。
- gamma: 控制 pretraining gradient mix 强度。
- gamma = 0 时就是 PPO。
- 论文默认 InstructGPT 多数情况下指 PPO-ptx。

为什么不只是调大 KL? Appendix 的实验显示, 单纯增大 KL coefficient 会显著降低 validation reward, 而且不能完全修复 DROP、SQuAD 等退化。PPO-ptx 更像是在保留预训练能力的同时进行偏好优化。

10. 四模型视角

PPO-RLHF 训练时实际有四个模型角色:

actor / policy:

trainable, initialized from SFT

critic / value function:

estimates return, often shares backbone or initialized from RM

reference model:

frozen SFT, computes KL penalty

reward model:

frozen RM, gives final scalar score

这也是本地 `ppo_llm_minimal.py` 开头注释里的核心。调试 RLHF 时一定要问:

1. actor 是不是从 SFT 初始化?
2. ref 是不是 frozen?
3. RM 是不是 frozen?
4. KL 是按 token 加, 还是只在序列末尾加?
5. response mask 是否只覆盖回答部分?
6. old logprob 是否来自 rollout 时的 policy?

这些问题比“用了 PPO 这个词”更重要。

11. 张量级别走一遍

假设 batch size $B=2$, sequence length $T=12$:

```
input_ids:      [B, T]
response_mask:  [B, T]
actor logits:   [B, T, V]
ref logits:     [B, T, V]
log_p_act:      [B, T-1]
log_p_ref:      [B, T-1]
rm_score:       [B]
token_rewards:  [B, T-1]
values:         [B, T]
advantages:     [B, T-1]
returns:        [B, T-1]
```

PPO update:

```
ratio_t = exp(log_p_new_t - log_p_old_t)
```

```
policy_loss_t =
    - min(
        ratio_t * advantage_t,
        clip(ratio_t, 1 - eps, 1 + eps) * advantage_t
    )
```

Value loss:

$\text{value_loss} = \text{mean}((V_t - \text{return}_t)^2 \text{ over response tokens})$

Entropy bonus:

$\text{entropy} = - \sum_v p(v) \log p(v)$

这就是为什么 RLHF 代码比 SFT 难调得多: 你不只在算 cross entropy, 还要同时维护 rollout policy、old logprobs、reference KL、reward model score、value function、advantage normalization 和 response masks。

12. 实验证据链

论文不是只展示几个漂亮 demo, 它构造了多条证据:

- InstructGPT 更符合用户意图: 标注者在 API held-out prompts 上更偏好 InstructGPT。
- 尺寸不是全部: 1.3B InstructGPT 可胜过 175B GPT-3。
- RLHF 优于 SFT: 从 GPT-3 -> prompted GPT-3 -> SFT -> PPO 有阶梯式提升。
- 不只是训练标注者过拟合: held-out labelers 也偏好 InstructGPT。
- Reward model 有泛化: held-out labeler group 上 RM accuracy 约 69.6%, 训练组约 72.4%。
- Truthfulness 改善: TruthfulQA 和 closed-domain API tasks 中更少 hallucination。
- Toxicity 有改善但有限: RealToxicityPrompts 上有小幅改善, 但 bias 没有显著改善。
- Public NLP 有退化: PPO 会损伤部分传统 benchmark, PPO-ptx 缓解但不完全解决。

最重要的人评数字:

- 175B InstructGPT 相比 175B GPT-3, 标注者偏好约 85 +/- 3%。
- 175B InstructGPT 相比 few-shot 175B GPT-3, 偏好约 71 +/- 4%。
- 1.3B InstructGPT 可被偏好于 175B GPT-3。

读这些实验时不要只记住 “RLHF 有用”。更准确的结论是: 在该论文的 API prompt distribution、该标注协议、该模型家族和该评价方式下, 人类偏好优化显著提高了 assistant 行为, 但仍存在安全、偏见、真实性和 benchmark regression 问题。

13. 为什么这篇论文能成为范式

它把 “对齐” 变成了一个工程闭环:

```
collect behavior data
train initial policy
collect preference data
train reward model
optimize policy
evaluate with humans
monitor regressions
iterate
```

这比单纯 instruction tuning 多一个关键环节: preference data。人类往往很难写出完美答案, 但比较两个答案哪个更好更容易。这种 “比较比生成更容易” 的思想, 是 RLHF 和 preference optimization 的认知基础。

它也把用户分布放到中心。论文的数据大量来自 API prompt, 而不是只来自静态 benchmark。这对现代 LLM 产品很关键: 好模型不是在某个榜单上最高分, 而是在真实用户请求上更可靠、更有用、更可控。

14. 局限和后续问题

论文自己承认了许多限制:

1. 对齐对象有限: 模型对齐到标注者和研究团队的偏好, 不代表全体用户或所有受影响群体。
2. 评价仍然主观: preference ratings 受标注说明、标注者背景、界面和比较集合影响。
3. Truthfulness 没有彻底解决: 模型仍会犯简单错误。
4. Toxicity 改善有限, bias 没有显著改善。
5. PPO 训练复杂且不稳定, 需要 KL、RM、value function、采样策略等多重监控。
6. RM 可能被 overoptimized, policy 可能学会 reward hacking。
7. Public NLP benchmark regression 说明 alignment 可能有能力迁移成本。
8. API 中心化部署带来治理和权力集中问题。

这些局限后来分别催生了很多研究:

- DPO: 直接从 preference data 优化 policy, 绕开显式 RM+PPO。
- Constitutional AI/RLAIF: 降低人类标注成本, 引入原则反馈。
- Process reward model: 不只评价最终答案, 也评价推理过程。
- Red teaming: 主动寻找 reward model 和 safety policy 的漏洞。
- Multi-objective alignment: 同时处理 helpfulness、truthfulness、harmlessness、fairness。

15. 与 DPO 的关系

DPO 可以从 InstructGPT 的 RLHF 目标中看出来。InstructGPT 显式训练 RM, 然后用 PPO 最大化:

`reward - beta * KL(policy || reference)`

DPO 问了一个反向问题: 如果 KL-regularized RL 的最优 policy 和 reward 之间存在解析关系, 能不能不训练显式 RM、不跑 PPO, 直接用 chosen/rejected pair 训练 policy?

所以理解 DPO 之前, 必须先理解 InstructGPT:

- DPO 的 reference policy 对应 InstructGPT 的 frozen SFT policy。
- DPO 的 preference pairs 对应 InstructGPT 的 RM training data。
- DPO 的 beta 对应 KL 控制强度。
- DPO 避免了 PPO rollout 和 value function, 但继承了 preference data 的偏差与覆盖问题。

16. 与本仓库代码怎么对上

本模块的最小代码基本对应论文三段:

- `src/sft_minimal.py`: 对应 Stage 1 SFT, 重点看 response-only NLL 和 `ignore_index=-100`。
- `src/rm_minimal.py`: 对应 Stage 2 RM, 重点看 LM + scalar head、last non-pad hidden 和 BT loss。
- `src/common.py`: 对应 preference helpers, 重点看 `bt_loss`、padding 和 masks。
- `src/ppo_llm_minimal.py`: 对应 Stage 3 PPO, 重点看 actor/critic/ref/RM、token-level KL reward 和 PPO clip。
- `src/reward_hacking_demo.py`: 对应 RLHF failure mode, 重点看 reward proxy 如何被利用。
- `src/capstone_tldr_rlhf.py`: 对应 toy RLHF pipeline, 重点看如何用小模型复刻三阶段。

学习时建议这样跑脑内 debugger:

1. 给一个 prompt, 让 SFT 生成 4 个候选。
2. 人类排序得到 $A > C > B > D$ 。
3. 展开成 pair: (A,C), (A,B), (A,D), (C,B), (C,D), (B,D)。

4. RM 对每个 completion 给一个标量。
5. BT loss 推高 preferred, 压低 rejected。
6. PPO 用 RM 分数推 actor, 同时用 reference KL 拉住 actor。

17. 极简代码: 从偏好对到 PPO reward

Reward model loss:

```
import torch
import torch.nn.functional as F

r_chosen = torch.tensor([2.0, 0.4, 1.1])
r_rejected = torch.tensor([0.3, 0.7, -0.2])

loss = -F.logsigmoid(r_chosen - r_rejected).mean()
acc = (r_chosen > r_rejected).float().mean()

print(loss.item(), acc.item())
```

Token-level KL reward:

```
beta = 0.02
log_p_actor = torch.tensor([[ -1.0, -0.8, -0.5]])
log_p_ref = torch.tensor([[ -1.1, -0.7, -0.6]])
rm_score = torch.tensor([1.2])

kl = log_p_actor - log_p_ref
token_rewards = -beta * kl
token_rewards[0, -1] += rm_score[0]

print(token_rewards)
```

如果你真的理解了这两段, 就能看懂大多数 RLHF 训练日志:

- RM loss 降, 不代表 policy 一定好。
- RM accuracy 高, 不代表 reward 不会被 hack。
- KL 太低, policy 没学动。
- KL 太高, policy 可能跑偏。
- reward 升高但人评下降, 是典型 proxy failure 信号。

18. 现在为什么还要读

今天读 InstructGPT, 不是为了照搬 PPO 训练 ChatGPT。现实工程里, 很多团队会用 SFT、DPO、IPO、KTO、RLAIF、rejection sampling、best-of-n、online preference learning 等替代或组合路线。但这篇论文仍是根:

- 它定义了 assistant alignment 的问题形式。
- 它展示了真实 prompt distribution 的重要性。
- 它把 human preference 变成可训练信号。
- 它揭示了 KL reference model 的必要性。
- 它提前暴露了 reward hacking、偏好代表性、benchmark regression、安全评价等问题。

对学习者来说, 这篇论文是“LLM 产品行为为什么不是纯预训练能力”的最佳入口。

19. 常见误区

1. 误区: RLHF 就是让模型更聪明。
 - 更准确: RLHF 改变模型行为分布, 使它更符合标注偏好; 它可能提升用户体验, 但也可能损伤某些 benchmark。
2. 误区: Reward model 是真理函数。
 - 更准确: RM 是标注偏好的代理模型, 会有偏差、过拟合和可被利用的漏洞。
3. 误区: KL penalty 越大越安全。
 - 更准确: KL 太大会压住学习, 太小会 reward hacking; 单纯调大 KL 也不能完全修复能力退化。
4. 误区: 人类偏好就是全人类价值。
 - 更准确: 论文对齐的是特定标注者在特定说明下的偏好。
5. 误区: SFT 不重要, PPO 才是主角。
 - 更准确: SFT 是 PPO 的初始 policy 和 reference anchor, SFT 质量决定后续优化空间。

20. 闭卷掌握检查

1. 为什么 GPT-3 变大不自动等于更会 follow instructions?
2. InstructGPT 三阶段分别用什么数据、训练什么模型、优化什么 loss?
3. 为什么 SFT loss 只算 response tokens?
4. Reward model 为什么输出 scalar, 而不是每个 token 一个分类标签?
5. Bradley-Terry loss 的公式是什么?
6. 标注者对 K 个回答排序时, 为什么不能把所有 pair 完全当独立样本?
7. PPO 阶段为什么要 frozen reference model?
8. $\text{reward} - \beta * \text{KL}$ 里的 β 太大或太小分别会怎样?
9. PPO-ptx 解决了什么问题? 为什么不是单纯调大 KL?
10. 为什么 1.3B InstructGPT 胜过 175B GPT-3 是一个范式信号?
11. 这篇论文的 alignment claim 有哪些边界?
12. DPO 如何继承并改写 InstructGPT 的 RLHF 目标?

21. 用 AI agent 学这篇的正确方式

不要让 agent 直接“总结 InstructGPT”。那样只会得到三段式口号。更好的学习 prompt 是:

我正在读 InstructGPT。请你按 Figure 2 带我画出 SFT/RM/PPO 三阶段。

每到一阶段, 先问我输入输出是什么, 再让我写出 loss。

然后用一个 $\text{batch size}=2, T=8$ 的例子让我标注所有 tensor shape。

最后用论文实验结果检查每个设计到底被什么证据支持。

如果我说“RLHF 对齐人类价值”, 请纠正为“对齐特定标注者偏好”并解释边界。

真正掌握这篇论文的标志是: 你能从 next-token prediction 的错位出发, 解释为什么需要示范、排序和 KL-regularized policy optimization; 能手写 BT loss 和 token-level KL reward; 能说清楚 1.3B 胜过 175B 的证据含义; 也能指出这套方法的偏好代表性、reward hacking 和能力退化问题。